

Enatega Store App — Security, Performance & Code Cleanup Audit

Repository: Enatega Store App **Module:** enatega-multivendor-store **Branch:** adeel **App type:** Store/vendor React Native (Expo) mobile application **Audit date:** 2026-06-29 **Auditor:** Automated code audit (evidence-based, repository-only)

Executive Summary

The Enatega Store App is an Expo Router / React Native vendor application for restaurant/store owners to manage incoming orders, accept/reject orders, set preparation time, print thermal receipts, toggle availability, manage work schedules, view earnings, and request wallet withdrawals. It connects to a **proprietary GraphQL backend** (<https://aws-server-v2.enatega.com/graphql>) that is **not present in this repository**, so all backend authorization logic is out of scope and is explicitly marked *Backend verification required* where relevant.

The audit surfaced **several Critical, repository-verifiable security problems**, dominated by **committed secrets**:

- A **Firebase Admin SDK private key** (`google-service-account.json`) is committed and tracked in git.
- A **Google Play publishing service-account private key** (`google-service-account-copy.json`) is committed and is **not even listed in .gitignore**.
- A **Sentry auth token** is hardcoded in `eas.json`.
- The **store's plaintext password is fetched from the API and rendered on the profile screen**.

Beyond secrets, the app trusts the locally stored `store-id` as the authorization key for earnings/withdrawal/profile/availability operations (classic BOLA/IDOR surface, backend-dependent), logs sensitive data to the console, and has no real auth guard on the protected route group.

On **performance**, the order feed polls the full order list every 5 seconds with `network-only` (no cache), and — because of a storage-key mismatch (`restaurantId` vs `store-id`) — the real-time WebSocket order subscription **never actually subscribes**, leaving polling as the only mechanism. A chat subscription re-subscribes on every render, and a list effect can trigger a refetch loop.

On **cleanup**, large portions of the GraphQL layer are copy-pasted leftovers from the customer/rider apps and are entirely unused (6 mutation files, several individual operations, a duplicate orders query), 31 translation bundles are eagerly loaded though only ~7 are user-selectable, and roughly a dozen dependencies are installed but never imported.

Overall risk rating: CRITICAL — driven almost entirely by committed private keys and

credential exposure that are directly verifiable in this repository.

Scope

In scope (analyzed):

- All application source in `app/` and `lib/`
- Config files: `app.json`, `eas.json`, `environment.ts`, `metro.config.js`, `babel.config.js`, `tsconfig.json`, `eslint.config.mjs`, `.gitignore`, `.easignore`, `.npmrc`
- Committed credential files: `google-services.json`, `google-service-account.json`, `google-service-account copy.json`
- `package.json` / dependency surface
- GraphQL queries/mutations/subscriptions under `lib/apollo/` and `lib/api/graphql/`
- `i18n` (`i18next.ts`, `languages/`)

Out of scope (not in repository):

- The proprietary GraphQL backend / API resolvers and all server-side authorization, validation, and rate-limiting
 - Native build artifacts, the EAS cloud build environment
 - `package-lock.json` transitive CVE verification via `live npm audit` (network was unavailable during this audit; see *Dependency Security*)
-

Methodology

1. **Static structure mapping** — enumerated routes, screens, components, hooks, contexts, services, and the GraphQL layer.
2. **Manual code reading** — read the auth/session, Apollo client, public-token service, order/earnings/withdraw flows, notifications, printing, and `i18n` in full.
3. **Pattern search** — grepped for secrets (`AIza`, `-----BEGIN`, `sntrys_`, `password`, `token`), `console.*`, `AsyncStorage/SecureStore`, `fetchPolicy`, `pollInterval`, list rendering, and dynamic URLs.
4. **Dead-code/dependency verification** — cross-referenced every suspect export/package against actual import sites (`grep -r l`) to confirm zero usage before flagging.
5. **Git tracking checks** — `git ls-files`, `git log` to confirm whether ignored-looking files are actually tracked and historically committed.

Every finding below cites a real file, line(s), and snippet. Items that depend on the unseen backend

are labeled accordingly.

Project Structure Map

Platform / tooling

Item	Value	Source
Expo SDK	^54.0.0	package.json
React Native	0.81.5	package.json
React	19.1.0	package.json
Router	expo-router ~6.0.15 (typed routes)	package.json, app.json
New Architecture	newArchEnabled: true	app.json
Language	TypeScript ~5.9.2 (strict)	tsconfig.json
Styling	NativeWind 4 + Tailwind 3	metro.config.js, tailwind.config.js
State	React Context (no Redux)	lib/context/global/*
GraphQL	Apollo Client ^3.14.0 + subscriptions- transport-ws WS link	lib/apollo/index.ts
i18n	i18next + react- i18next	i18next.ts
Tests	jest + jest-expo preset (no test files found)	package.json

Navigation (Expo Router file-based)

app/ _layout.tsx	Root: Apollo, theme, internet, config, auth,
user providers index.tsx	Entry redirect (token → home / login) +
push-token bootstrap +not-found.tsx (un-protected)/ _layout.tsx login/index.tsx (protected)/ _layout.tsx	Wraps Restaurant + Sound providers (NO auth check)
(tabs)/_layout.tsx home/	Orders (new / processing / delivered),
language, profile, earnings/ earnings-order-details profile/ wallet/ order-detail/index.tsx	bank-management, orders, help, work-schedule Earnings summary + earnings-detail / Wallet + withdraw success route

lib/ modules

Path	Purpose
lib/apollo/index.ts	setupApollo() — HTTP + deprecated WS split, auth middleware
lib/apollo/queries/*, lib/apollo/mutations/*	GraphQL ops (mix of used + leftover)
lib/api/graphql/*	Second GraphQL op tree (login, restaurant, order, notification, configuration)
lib/context/global/ auth.context.tsx	Token (SecureStore), logout, language bootstrap
lib/context/global/ restaurant.tsx	Orders query (poll 5s) + order subscription
lib/context/global/ user.context.tsx	Store profile (STORE_PROFILE) by store-id
lib/context/global/ configuration.context.tsx	Server-driven config (currency, Sentry URL)
lib/context/global/ sound.context.tsx	New-order alert sound (expo-av)
lib/services/public-access- token.service.ts	"bop-auth" public token + nonce
lib/services/async-storage.ts	AsyncStorage wrapper + event emitter
lib/hooks/*	useLogin, useOrders, useAcceptOrder, useCancelOrder, useOrderPickedUp, useOrderRing, useChat, useNotification, usePrintOrder, useQueryQL, useLazyQueryQL
lib/ui/screens, lib/ui/screen- components	Screen + sub-components
lib/utils/*	constants, interfaces, methods, schema (Yup), enums

Auth / session

- JWT stored in **SecureStore** under key store-token (STORE_TOKEN); store-id stored in **AsyncStorage**.
- Public "bop-auth" token + nonce stored in **SecureStore** under obfuscated keys (usr_prf_cache, dev_meta_id, sess_ttl_ts).
- Login via restaurantLogin mutation; demo credentials prefilled via

lastOrderCreds query.

- No signup/OTP/forgot-password flow wired in the store app (leftover customer mutations exist but are unused).

Config / build files

app.json, eas.json, environment.ts, google-services.json, google-service-account.json, google-service-account copy.json, metro.config.js, babel.config.js.

Findings Count Table

Severity	Security	Performance	Cleanup	Total
Critical	4	0	0	4
High	4	2	1	7
Medium	6	6	4	16
Low	3	3	5	11
Info	2	1	3	6
Total	19	12	13	44

Top Security Risks

Priority	Issue	Impact	Location	Effort
1	Firebase Admin SDK private key committed	Full backend/Firebase project compromise	google-service-account.json:5	Low (rotate + purge)
2	Google Play publishing key committed (not git-ignored)	Malicious app uploads to Play Store	google-service-account copy.json:5	Low (rotate + purge)
3	Store plaintext password fetched + shown in UI	Credential disclosure, account takeover	store.query.ts:76, documents/index.tsx:135	Medium
4	Sentry auth token hardcoded in build config	Sentry org/project tampering, source-map exfil	eas.json:22	Low (rotate)
5	Firebase API key + google-services.json	Abuse of Firebase project quotas/services	google-services.json:24, .gitign	Low

Priority	Issue	Impact	Location	Effort
	n committed (gitignore typo)		ore:8	
6	store-id from AsyncStorage trusted as authz key	BOLA/IDOR on earnings/withdraw/profile	wallet/view/main/index.tsx:182, drawer-header/index.tsx:43	Backend
7	Demo store credentials auto-fetched + prefilled	Public access to a live demo store account	mutation/login/index.ts:20, useLogin.ts:47	Medium
8	Sensitive data (username, tokens, PII) logged	Credential/PII leakage via logs/crash tooling	useLogin.ts:72, 123, app-wide	Low

Top Performance Wins

Priority	Issue	Estimated Impact	Location	Effort
1	Order list polls every 5s with network-only	Constant battery/network/server load	restaurant.tsx:31-38	Low
2	Realtime order subscription never subscribes (key mismatch)	Forces reliance on aggressive polling	restaurant.tsx:73	Low
3	Chat subscribeToMore effect has no deps array	Re-subscribes every render (leak)	useChat.tsx:66-78	Low
4	orders.length === 0 effect calls refetch()	Refetch loop with polling	new-orders.tsx:104-109	Low
5	31 translation bundles eagerly imported	Larger bundle + startup memory	i18next.ts:8-73	Medium
6	Hooks called after early return null	Rules-of-Hooks violation / re-render churn	order/index.tsx:34, 46	Medium

Top Cleanup Opportunities

Priority	Code to remove/refactor	Why	Location	Removal risk
1	6 unused GraphQL mutation files	Customer/rider leftovers, never imported	lib/apollo/mutations/*	Safe
2	~11 unused npm dependencies	Never imported; bloat + supply-chain surface	package.json	Medium
3	Duplicate orders query STORE_ORDERS	Identical to GET_ORDERS, only in a comment	store.query.ts	Safe
4	Unused mutations within live files	ASSIGN_ORDER, reviewOrder, placeOrder, ...	order.mutation.ts, rider.mutation.ts	Safe
5	Backup/empty files	*.backup, empty dummy/index.ts	see below	Safe

PART 2 — Security Findings

[CRITICAL] Firebase Admin SDK private key committed to the repository

Category: Security / Storage / Sensitive Data Exposure **Location:** google-service-account.json:1-13 (private key at line 5) **Description:** A complete Google Cloud **service-account private key** for the Firebase Admin SDK (firebase-adminsdk-oyyl6@enatega-multivendor.iam.gserviceaccount.com, project enatega-multivendor) is committed. .gitignore:8 lists google-service-account.json, but the file is **already tracked** (the ignore rule does not untrack files added earlier), confirmed via `git ls-files` and `git log` (committed in ba6e07b0 added new rider and store app, modified in f77ecb30). **Evidence:**

```
// google-service-account.json
"private_key": "-----BEGIN PRIVATE KEY-----
\nMIIEVgIBADANBgkqhkiG9w0BAQEFAASCBAgEAAoIBAQDJew4azRPdrd1y...-----END
PRIVATE KEY-----\n",
"client_email": "firebase-adminsdk-oyyl6@enatega-
multivendor.iam.gserviceaccount.com",
```

```
$ git ls-files | grep google-service
google-service-account copy.json
google-service-account.json
```

Impact: An Admin SDK key grants **full administrative access** to the Firebase/GCP project — read/write to all data, sending arbitrary push notifications to every app user, minting custom auth tokens (impersonating any user), and consuming billable services. Anyone with repository (or git-history) access has complete backend compromise. **Reproduction / How to verify:**

1. `git ls-files | grep google-service-account.json` → file is tracked.
 2. `git log --oneline -- google-service-account.json` → shows it has been in history since `ba6e07b0`.
 3. Open the file → full RSA private key present. **Recommended fix:**
 4. **Immediately revoke/rotate** this key in Google Cloud IAM and the Firebase console.
 5. Remove from the working tree and **purge from git history** (`git filter-repo --path google-service-account.json --invert-paths`, then force-push and rotate again).
 6. Provision the key only in the EAS build environment via **EAS secrets** (`eas secret:create`), never in the repo.
 7. Fix `.gitignore` (see HIGH finding on the typo) and add a pre-commit secret scanner (e.g. `gitLeaks`). **References:** OWASP MASVS-STORAGE-1, OWASP Mobile Top 10 M9 (Insecure Data Storage)/M1, CWE-798 (Use of Hard-coded Credentials), CWE-312.
-

[CRITICAL] Google Play publishing service-account key committed (and NOT git-ignored)

Category: Security / Storage / Sensitive Data Exposure **Location:** `google-service-account-copy.json:1-12` (private key at line 5) **Description:** A second service-account private key — `android-playstore-eas@pc-api-8741271665364646084-744.iam.gserviceaccount.com` — is committed. This is the **Google Play Android Publisher** service account used by `eas submit`. Unlike the first file, **this filename is not present in `.gitignore` at all**, so it will continue to be committed. **Evidence:**

```
// google-service-account copy.json
"private_key": "-----BEGIN PRIVATE KEY-----
\nMIIEvQIBADANBgkqhkiG9w0BAQEFAASCBCwggSjAgEAAoIBAQCyl3+dsZuMKn8K...-----END
PRIVATE KEY-----\n",
"client_email": "android-playstore-eas@pc-api-8741271665364646084-
744.iam.gserviceaccount.com",
```

`.gitignore` only contains `google-services.json` (typo) and `google-service-account.json` — there is no `*copy*` entry. **Impact:** A Play Android Publisher key allows **uploading new app binaries/releases** and editing the store listing for the developer account. A leaked key enables **supply-chain compromise**: pushing a malicious update to all store users. **Reproduction / How to verify:**

1. `git ls-files | grep "copy"` → `google-service-account copy.json` is tracked.

2. `grep -n copy .gitignore` → no match (file is not ignored). **Recommended fix:** Revoke this key in Google Cloud IAM, delete the file, purge from history, and store the publishing key as an EAS secret. Add `google-service-account*.json` (glob) to `.gitignore`. **References:** CWE-798, CWE-312, OWASP MASVS-STORAGE-1, supply-chain (SLSA) risk.
-

[CRITICAL] Store plaintext password fetched from API and rendered in the UI

Category: Security / Sensitive Data Exposure / GraphQL over-fetch **Location:**

`lib/apollo/queries/store.query.ts:76` (query) → `lib/ui/screen-components/profile/view/docs/documents/index.tsx:135-138`

(display) **Description:** The `STORE_PROFILE` query explicitly requests the restaurant's password field, and the profile "documents" screen renders it as readable text. The backend is returning a usable password value to the client. **Evidence:**

```
// store.query.ts (STORE_PROFILE)
restaurant(id: $restaurantId) {
  ...
  username
  password          // ← credential fetched to the device
  ...
}

// profile/view/docs/documents/index.tsx:133-138
<Text ...>{t("Password")}</Text>
{dataProfile?.password ? (
  <Text style={{ color: appTheme.fontSecondColor }}>
    {dataProfile?.password}      // ← password shown on screen
  </Text>
) : ( ... )}
```

Impact: The store credential is transmitted to and **cached in the Apollo InMemoryCache** and shown in plaintext. Anyone with brief physical access (shoulder-surf), a screenshot, a memory/cache dump, or device-level debugging obtains the account password. It also implies the backend stores recoverable (non-hashed) passwords. **Reproduction / How to verify:**

1. Log in, open Profile → Documents; the password is displayed.
 2. Inspect the network response for the `Restaurant` query → `password` is present in the payload. **Recommended fix:** Remove password from `STORE_PROFILE` (and any other query). Never return passwords from the API; the backend should store only salted hashes. Remove the password row from the profile UI. **References:** OWASP Mobile Top 10 M9/M2, MASVS-STORAGE-1/AUTH, CWE-256 (Plaintext Storage of a Password), CWE-359, CWE-200.
-

[CRITICAL] Sentry auth token hardcoded in `eas.json`

Category: Security / Sensitive Data Exposure / CI config **Location:** `eas.json:22` **Description:**

A Sentry auth token (sntrys_...) is committed in the staging build profile's env. **Evidence:**

```
// eas.json (build.staging.env)
"SENTRY_AUTH_TOKEN":
"sntrys_eyJpYXQiOiE3NDAwNTQwNDYuNzcxMjM4...G2Tc7Rxn8eaEjd0SpjCyEqaeCo6EwK5qG411N
aBZKwY"
```

Impact: Sentry auth tokens can read/modify projects, upload/download source maps (potential source-code exposure), and manage org resources for ninjas-code-w7. Valid until revoked. **Reproduction / How to verify:** `grep -n SENTRY_AUTH_TOKEN eas.json`. **Recommended fix:** Revoke the token in Sentry, remove it from eas.json, and inject via EAS secrets (`eas secret:create --name SENTRY_AUTH_TOKEN`). Purge from history. **References:** CWE-798, CWE-312, OWASP MASVS-STORAGE-1.

[HIGH] Firebase google-services.json committed; .gitignore rule typo'd

Category: Security / Sensitive Data Exposure **Location:** google-services.json:24,68,104; .gitignore:8 **Description:** google-services.json (containing the Firebase Android API key and OAuth client IDs) is committed. .gitignore intends to ignore it but contains the typo google-servicees.json (extra "e"), so the real file was never ignored. **Evidence:**

```
// google-services.json
"api_key": [ { "current_key": "AIzaSyA-Arn9vDqHen1EJimV8lJfNsY0BLNcIfg" } ]

# .gitignore
google-servicees.json      ← typo; does not match google-services.json
google-service-account.json
```

Impact: Android Firebase API keys are technically distributed inside the APK, but committing the file plus the gitignore typo signals broader secret-hygiene failure and allows quota abuse if Firebase API restrictions are not configured. Note the same key is reused across com.enatega.multirider, com.enatega.multivendor, and multivendor.enatega.restaurant. **Reproduction / How to verify:** `git ls-files | grep google-services.json` (tracked); `grep -n servicees .gitignore` (typo present). **Recommended fix:** Fix the gitignore entry to google-services.json (and add google-service-account*.json). Restrict the Firebase API key to the specific apps/APIs in the Google Cloud console. Provision via EAS at build time. **References:** CWE-312, OWASP MASVS-STORAGE-1.

[HIGH] Demo/last-order store credentials auto-fetched and prefilled into login

Category: Security / Auth **Location:** lib/api/graphql/mutation/login/index.ts:20-27,

lib/hooks/useLogin.ts:32,47-56, lib/ui/screens/login/index.tsx:53-61 **Description:** On the login screen, useLogin runs the lastOrderCreds query, which returns restaurantUsername and restaurantPassword; these are written into the form's initial values, prefilling a working store account for anyone who opens the app. **Evidence:**

```
// mutation/login/index.ts
export const DEFAULT_STORE_CREDS = gql`
  query LastOrderCreds { lastOrderCreds { restaurantUsername
restaurantPassword } }`;

// useLogin.ts:47-56
} else if (lastOrderCreds?.restaurantUsername &&
lastOrderCreds?.restaurantPassword) {
  setCreds({ username: lastOrderCreds.restaurantUsername,
              password: lastOrderCreds.restaurantPassword });
}
```

Impact: A live store account's credentials are served to any unauthenticated client and auto-filled, granting access to that store's orders, earnings, and withdrawal flow. Even if intended as a demo convenience, it is real credential exposure over the network. **Reproduction / How to verify:** Fresh install → open Login; observe the username/password fields prefilled. Inspect the LastOrderCreds network response. **Recommended fix:** Remove the demo auto-fill from production builds (gate behind __DEV__ or a build flag), and have the backend stop returning a real password. If a demo is required, use a clearly-scoped sandbox account with least privilege and no withdrawal capability. **References:** OWASP MASVS-AUTH-1, CWE-798, CWE-200.

[HIGH] Authorization keyed on client-controlled store-id (BOLA/IDOR surface)

Category: Security / Authorization (BOLA) **Location:**

lib/context/global/user.context.tsx:63-66, lib/ui/screen-components/wallet/view/main/index.tsx:179-184, lib/ui/screen-components/home/drawer/drawer-header/index.tsx:43 **Description:** userId is read from AsyncStorage store-id and passed directly as the identifying argument to sensitive queries/mutations: STORE_PROFILE, STORE_BY_ID, STORE_EARNINGS, STORE_TRANSACTIONS_HISTORY, STORE_CURRENT_WITHDRAW_REQUEST, createWithdrawRequest(userId), and toggleStoreAvailability(restaurantId). The client trusts a value it stores locally rather than deriving the store identity from the authenticated JWT. **Evidence:**

```
// user.context.tsx:63-66
const getUserId = useCallback(async () => {
  const id = await AsyncStorage.getItem("store-id");
  if (id) setUserId(id);
}, [userId]);

// wallet/view/main/index.tsx:179-184
await createWithdrawRequest({
  variables: { requestAmount: withdrawAmount, userId: userId }, // userId from
store-id
});
```

Impact: If the backend authorizes by the supplied `userId/restaurantId` instead of the JWT subject, a vendor could edit `AsyncStorage (store-id)` on a rooted/jailbroken device or via a tampered build and **read another store's earnings/transactions, create withdrawals, or toggle another store's availability** — broken object-level authorization. **Mobile-side risk:** the app passes an attacker-controllable identifier as the authorization key. **Backend-dependent risk / Cannot be fully verified from this repo:** whether the server re-derives identity from the token and rejects mismatches. **Backend verification required. Reproduction / How to verify:**

1. Log in as Store A; read `store-id` from `AsyncStorage`.
2. Replace it with Store B's id (rooted device or instrumented build) and reload.
3. Observe whether earnings/transactions/withdraw operate on Store B. If yes → BOLA. **Recommended fix:** Backend must ignore client-supplied store identifiers for owner-scoped operations and use the authenticated principal. Client-side, stop sending `userId/restaurantId` where the server can infer them; treat `store-id` as a UI hint only. **References:** OWASP API Security Top 10 API1:2023 (BOLA), OWASP Mobile M1/M3, CWE-639, CWE-285.

[HIGH] Sensitive data (credentials, tokens, PII) written to console logs

Category: Security / Logging **Location:**

`lib/hooks/useLogin.ts:39,72,85,101,123,144; lib/services/public-access-token.service.ts:72,108,129;`

`lib/context/global/auth.context.tsx:44,54; app/index.tsx:56; 68`

`console.*` calls total across `lib/ + app/` **Description:** Login and bootstrap paths log the username, notification/push tokens, full permission objects, and full login mutation results. Order data carrying customer PII (name, phone, email) flows through screens that also log errors. ESLint does not forbid `console` (no `no-console` rule), so these ship in release builds unless stripped. **Evidence:**

```
// useLogin.ts:72
console.log("□ ~ onLogin called with:", { username, password: "****" }); //
username in cleartext
// useLogin.ts:123
console.log("□ ~ Got push token:", notificationToken);
// useLogin.ts:144
console.log("□ ~ Login mutation result:", data); //
token + restaurantId

// public-access-token.service.ts:129
console.log("□ Token refreshed, expires at:", new
Date(expiryTime).toISOString());
```

Impact: On Android, `console.*` reaches logcat and can be read by other processes/tools and by crash/analytics SDKs. Logged usernames, push tokens, and JWTs enable account/notification abuse; logged order objects leak customer PII. **Reproduction / How to verify:** Run a device build, `adb logcat | grep "□"`, perform a login → username, push token, and login result

appear. **Recommended fix:** Remove sensitive logs; add no-console (allow warn/error only) to ESLint; strip remaining logs in production via babel-plugin-transform-remove-console. Never log tokens, credentials, or order PII. **References:** OWASP MASVS-STORAGE-2, Mobile M9, CWE-532 (Insertion of Sensitive Information into Log File).

[MEDIUM] No authorization guard on the (protected) route group

Category: Security / Auth **Location:** app/(protected)/_layout.tsx:15-47; redirect logic only in app/index.tsx:25-34 **Description:** The protected layout renders providers and a Stack but performs **no token check**. Gatekeeping relies solely on the initial redirect in app/index.tsx. There is no re-validation on resume, no auto-logout on token expiry, and checkAuth() in auth.context only runs at mount. **Evidence:**

```
// app/(protected)/_layout.tsx – no token/redirect guard
export default function ProtectedLayout() {
  const { appTheme, currentTheme } = useApptheme();
  return (<RestaurantProvider.Provider><SoundProvider> ...
</SoundProvider></RestaurantProvider.Provider>);
}
```

Impact: Defense-in-depth gap. If any navigation path reaches a protected route without passing through index.tsx (deep link, programmatic navigation, future refactor), protected screens render without an auth check. Expired tokens are not proactively handled client-side. **Reproduction / How to verify:** Delete store-token from SecureStore while on a protected tab; the UI does not redirect to login until a manual reload triggers checkAuth. **Recommended fix:** Add an explicit guard in (protected)/_layout.tsx using AuthContext.token (<Redirect href="/(un-protected)/login" /> when absent), and trigger logout() on UNAUTHENTICATED GraphQL errors in the Apollo error link. **References:** OWASP MASVS-AUTH-2, Mobile M1.

[MEDIUM] Credentials/PII persisted in Apollo cache and SecureStore-as-storage

Category: Security / Storage **Location:** lib/apollo/index.ts:32, 37-40; lib/context/global/user.context.tsx:48-62 **Description:** STORE_PROFILE (incl. password, full bussinessDetails bank fields) and order lists (customer name/phone/email) are stored in the default InMemoryCache. While in-memory (not persisted to disk here), the password/bank data remain readable via React DevTools/Apollo DevTools and memory inspection. **Evidence:**

```
// store.query.ts STORE_PROFILE
bussinessDetails { bankName accountNumber accountName accountCode }
```

Impact: Sensitive financial and credential fields are broadly available to anything that can read the JS heap or cache. **Reproduction / How to verify:** Attach Apollo DevTools; inspect the normalized

Restaurant entity → password and bank details present. **Recommended fix:** Stop fetching password; fetch bank details only on the screen that needs them, and consider `fetchPolicy` scoping. Avoid caching financial fields globally. **References:** CWE-312, OWASP MASVS-STORAGE-1.

[MEDIUM] Over-fetching customer PII in order list and subscription

Category: Security / GraphQL **Location:** `lib/apollo/queries/orders.ts:63-68`, `lib/apollo/subscriptions.ts` (`user { _id name phone email }`) **Description:** Every order in `restaurantOrders` and every `subscribePlaceOrder` event includes the customer's name, phone, and email. The store UI primarily needs name + masked phone for fulfillment. **Evidence:**

```
user { _id name phone email }
```

Impact: Larger payloads (perf) and broader customer-PII exposure on-device and in logs/cache than the UI requires; a tampered client can harvest customer contact data in bulk. **Reproduction / How to verify:** Inspect the `Orders` response — email/phone present for all customers regardless of screen need. **Recommended fix:** Request only required fields; have the backend mask phone/email for vendor scope unless explicitly needed for an active order. **References:** OWASP API3:2023 (Excessive Data Exposure), CWE-213.

[MEDIUM] Deprecated subscriptions-transport-ws carries auth tokens over WebSocket

Category: Security / Dependency / GraphQL **Location:** `lib/apollo/index.ts:11, 24-31`; `package.json` (`subscriptions-transport-ws ^0.11.0`) **Description:** The WS link uses `@apollo/client/link/ws` (`WebSocketLink`), which depends on the **deprecated and unmaintained** `subscriptions-transport-ws` (last meaningful release 2021). The modern `graphql-ws` is even listed in `package.json` (`^6.0.6`) but is **not used**. **Evidence:**

```
import { WebSocketLink } from "@apollo/client/link/ws";
const wsLink = new WebSocketLink({ uri: WS_GRAPHQL_URL, options: { reconnect:
true, lazy: true, timeout: 30000 } });
```

Impact: An unmaintained transport will not receive security fixes. (Note: as flagged in the Performance section, the subscription is also misconfigured and never actually subscribes.) **Reproduction / How to verify:** `grep -rn "graphql-ws" lib app` → no usage; the WS link imports the deprecated package. **Recommended fix:** Migrate to `GraphQLWsLink` from `@apollo/client/link/subscriptions` backed by `graphql-ws` (already installed) and remove `subscriptions-transport-ws`. **References:** OWASP MASVS-CODE-2 (outdated components), CWE-1104.

[MEDIUM] Single hardcoded production GraphQL endpoint for all environments

Category: Security / Config **Location:** `environment.ts:13-26` **Description:** `getEnvVars` returns the **same production URLs** for production, staging, and the default branch, and ignores the `ConfigurationContext` it reads. There is no isolation between staging and production traffic. **Evidence:**

```
if (env === "production" || env === "staging") {
  return { GRAPHQL_URL: "https://aws-server-v2.enatega.com/graphql",
    WS_GRAPHQL_URL: "wss://aws-server-v2.enatega.com/graphql" };
}
return { GRAPHQL_URL: "https://aws-server-v2.enatega.com/graphql",
  WS_GRAPHQL_URL: "wss://aws-server-v2.enatega.com/graphql" };
```

Impact: Test/staging builds hit production data; no way to point internal builds at a safe environment. (Endpoint disclosure itself is expected for a client app.) **Reproduction / How to verify:** Read `environment.ts` — both branches are identical. **Recommended fix:** Use distinct staging/production endpoints per EAS channel; remove the unused `useContext(ConfigurationContext)` call (it is also a Rules-of-Hooks anti-pattern in a non-component function). **References:** CWE-1188 (Insecure Default), config hygiene.

[MEDIUM] Missing disabled state on critical action buttons → duplicate order mutations

Category: Security / Order Flow **Location:** `lib/ui/useable-components/custom-continue-button/index.tsx:12-27`; `lib/ui/useable-components/set-order-accept-time/index.tsx:48-59, 122-129` **Description:**

`CustomContinueButton` swaps its label for a spinner while `isLoading` but **never sets disabled** on the `TouchableOpacity`, so `onPress` remains active. The "Accept order" handler has no local in-flight guard either; rapid double-taps can fire `acceptOrder/muteRing` twice. **Evidence:**

```
// custom-continue-button/index.tsx
<TouchableOpacity {...props} className="..." style={{ backgroundColor:
  appTheme.primary }}>
  {isLoading ? <CustomSpinner /> : <Text ...>{title}</Text>}
</TouchableOpacity> // no disabled={isLoading}, onPress still fires
```

Impact: Duplicate accept/status mutations and duplicate withdrawal submissions (the wallet "Continue" uses the same button). Depending on backend idempotency, this can double-process an order/withdrawal. **Backend idempotency verification required.** **Reproduction / How to verify:** On the Accept sheet, double-tap "Done" quickly → two `acceptOrder` network calls. **Recommended fix:** Add `disabled={isLoading}` (and `activeOpacity/visual disabled` state) to `CustomContinueButton`; guard handlers with an in-flight ref; ensure the backend treats status transitions idempotently. **References:** CWE-362 (race), OWASP Mobile M7 (poor code quality).

[LOW] Customer/auth GraphQL operations leftover from other apps

Category: Security / Dead Code **Location:**

`lib/apollo/mutations/authentication.mutation.ts`

(`login/resetPassword/changePassword/OTP/Deactivate`),

`lib/apollo/mutations/user.mutation.ts` (`createUser/updateUser`) **Description:** Full credential and account-management mutations from the customer app are present but unused. They widen the apparent attack/operation surface and confuse reviewers about what the store app can do. **Impact:** No direct runtime exploit (never invoked), but increases the chance a future change wires up a dangerous operation. See cleanup section for removal. **Recommended fix:** Delete the unused files. **Removal safety:** Safe (verified zero imports). **References:** CWE-561 (Dead Code).

[LOW] Apple ID and App Store IDs committed in `eas.json`

Category: Security / Sensitive Data Exposure (low) **Location:** `eas.json:40-41` **Description:** `submit.production.ios` includes `appleId`, `ascAppId`, and `appleTeamId`. **Evidence:**

`"appleId": "sharan.gohar@icloud.com", "ascAppId":`

`"1526672537"`. **Impact:** Low — these are identifiers, not secrets, but the Apple ID email aids targeted phishing of the publisher account. Prefer EAS secrets. **Recommended fix:** Move submit credentials/identifiers to EAS secrets or local-only `eas.json` not committed. **References:** CWE-200.

[INFO] "Obfuscated" secure-store keys are not a security control

Category: Security / Storage (info) **Location:** `lib/services/public-access-token.service.ts:22-27` **Description:** The public-token service stores token/nonce/expiry under deliberately misleading keys (`usr_prf_cache`, `dev_meta_id`, `sess_ttl_ts`) and the `metricsGeneral` GraphQL fields are obfuscated (`experience`, `hehe`, ...). This is security-by-obscurity; values are still in SecureStore and recoverable on a compromised device. **Impact:** None positive; can mislead maintainers. SecureStore (Keychain/Keystore) is the actual protection. **Recommended fix:** Use clear key names; rely on SecureStore + backend short token TTLs rather than obfuscation. **References:** OWASP MASVS-RESILIENCE (obscurity $\neq$ control).

[INFO] File/image upload, WebView, deep-link risks — not present

Category: Security / Upload / WebView / Deep Link **Location:** repo-wide search **Description:** Verified there is **no image/file upload** (no `ImagePicker.launch*`, no `FormData`, no `Cloudinary/S3`), **no WebView/HTML rendering** (`react-native-webview` is a dependency but unused), and **no dynamic/user-controlled deep links** — `Linking.openURL` targets are

hardcoded (enatega.com, WhatsApp/store URLs in help/view/main/index.tsx and drawer-content/index.tsx). app.json scheme is the default myapp. **Impact:** None — these common mobile risk classes are absent. (Removing the unused react-native-webview is a cleanup item.) **Reproduction / How to verify:** `grep -rn "WebView|launchImageLibrary|FormData|cloudinary" lib app` → no matches. **References:** N/A (negative finding documented for completeness).

PART 3 — Performance Findings

[HIGH] Order feed polls every 5s with network-only (no caching)

Category: Performance / GraphQL **Location:**

lib/context/global/restaurant.tsx:31-38 **Description:** The global orders query polls every 5 seconds and uses `fetchPolicy: "network-only"`, refetching the **entire** `restaurantOrders` payload (with nested items, addons, customer PII, rider) on every cycle and bypassing the cache. **Evidence:**

```
const { ... } = useQuery(GET_ORDERS, { fetchPolicy: "network-only",  
pollInterval: 5000, ... });
```

Impact: Continuous battery drain, mobile-data usage, and backend load — 12 full refetches/minute per active store, all day. With many menu items/addons per order the payload is large. **Reproduction / How to verify:** Network inspector on the orders screen → an `Orders` request every ~5s even with no changes. **Recommended fix:** Fix the WebSocket subscription (next finding) and rely on it for new orders; raise `pollInterval` substantially (e.g. 30–60s) as a fallback; use `cache-and-network`; request a slimmer order shape.

[HIGH] Real-time order subscription never subscribes (storage-key mismatch)

Category: Performance / GraphQL / Correctness **Location:**

lib/context/global/restaurant.tsx:71-96 (line 73) vs

lib/hooks/useLogin.ts:44,150 **Description:** `subscribeToMoreOrders` reads `AsyncStorage.getItem("restaurantId")`, but the store id is saved under the key **store-id** at login. The key `restaurantId` is never written anywhere in the codebase, so `restaurant` is always `null` and the function returns before subscribing. **Evidence:**

```
// restaurant.tsx:73  
const restaurant = await AsyncStorage.getItem("restaurantId"); // never set  
if (!restaurant) return;                                     // always returns  
  
// useLogin.ts:44 — actual key written  
await setItem("store-id", restaurantLogin?.restaurantId);
```

Grep: "restaurantId" as a storage key appears **only** at restaurant.tsx:73. **Impact:** The intended push-based order updates never arrive via subscription; the app depends entirely on the 5s polling above. This is both a functional bug (no realtime) and the root cause forcing aggressive polling. **Reproduction / How to verify:** Place a new order from the customer app; observe it appears only after the next poll, not instantly. Add a log inside subscribeToMore — it never runs. **Recommended fix:** Use store-id (the actual key) or, better, derive the id from useContext().userId. Then reduce/disable polling.

[HIGH] Chat subscribeToMore effect has no dependency array → re-subscribes every render

Category: Performance / React Native Render / Memory Leak **Location:**

lib/hooks/useChat.tsx:66-78 **Description:** The effect that calls

subscribeToMessages(...) has **no dependency array**, so it runs on every render, creating a new subscription each time. The returned unsubscribe cleans up the previous one, but rapid re-renders churn subscriptions and can stack listeners. **Evidence:**

```
useEffect(() => {  
  const unsubscribe = subscribeToMessages({ document:  
    SUBSCRIPTION_NEW_MESSAGE, ... });  
  return unsubscribe;  
}); // ← missing [orderId]
```

Impact: Repeated WS subscribe/unsubscribe cycles, extra server load, and potential duplicate message handling while the chat screen is open. **Reproduction / How to verify:** Add a log in the effect; type in the chat input (causes re-renders) → effect fires repeatedly. **Recommended fix:** Add [orderId] (or []) as the dependency array so it subscribes once per order.

[MEDIUM] orders.length === 0 effect triggers refetch() (refetch loop risk)

Category: Performance / React Native Render **Location:**

lib/ui/screen-components/home/orders/main/new-orders.tsx:104-109 **De**

scription: An effect calls refetch() whenever the local orders array length is 0. Combined with the 5s poll and the data → onInitOrders → setOrders cycle, an empty feed produces continuous refetches on top of polling. **Evidence:**

```
useEffect(() => { if (orders?.length === 0) refetch(); }, [orders?.length]);
```

Impact: Extra network calls whenever there are no active orders (the common idle state), compounding the polling cost. **Reproduction / How to verify:** Open the New Orders tab with no pending orders; watch repeated Orders requests beyond the 5s poll. **Recommended fix:** Remove this effect; rely on poll/subscription. If a manual refresh is desired, gate it behind pull-to-refresh (already present).

[MEDIUM] Hooks invoked after early return null (Rules of Hooks violation)

Category: Performance / React Native Render / Correctness **Location:** lib/ui/useable-components/order/index.tsx:27-49 **Description:** Order returns early (if (!order) return null; at line 34, if (!configuration) return null; at line 44) before calling useApptheme, useContext, useTranslation, useCancelOrder, useOrderPickedUp. Conditionally calling hooks breaks React's hook ordering guarantees. **Evidence:**

```
if (!order) return null; // line 34
const { appTheme } = useApptheme(); // line 39 (hook after return)
...
if (!configuration) return null; // line 44
const { cancelOrder } = useCancelOrder(); // line 46 (hook after conditional return)
```

Impact: Can cause "rendered fewer/more hooks than expected" crashes and unstable re-renders when order/configuration toggle. ESLint would normally catch this, but react-hooks is not even registered in eslint.config.mjs and exhaustive-deps is turned off. **Reproduction / How to verify:** Render an Order while configuration is briefly undefined, then defined → hook-count warning/crash in dev. **Recommended fix:** Call all hooks unconditionally at the top, then branch on order/configuration for render output. Add eslint-plugin-react-hooks with rules-of-hooks: error.

[MEDIUM] 31 translation bundles eagerly imported at startup

Category: Performance / Bundle / Startup **Location:** i18next.ts:8-73; languages/ (31 files) **Description:** All 31 language modules are statically imported and registered as i18next resources at startup, though the language selector exposes ~29 and the documented supported set is only ar, de, en, fr, km, zh. Each file is a large translation object loaded into memory immediately. **Evidence:**

```
import { en } from "../languages/en"; ... import { nl } from "../languages/nl"; //
31 imports
export const languageResources = { en:{translation:en}, ... nl:
{translation:nl} };
```

Impact: Increased JS bundle size and startup parse/memory cost for languages most users never select. **Reproduction / How to verify:** ls languages | wc -l → 31; all are imported in i18next.ts. **Recommended fix:** Ship only the officially supported locales, or lazy-load a locale on selection (i18next backend/dynamic addResourceBundle). Also fix i18next.init being called multiple times (getStoredLanguage + the iOS block, lines 82 and 102) — initialize once.

[MEDIUM] Pervasive network-only / no shared cache strategy

Category: Performance / GraphQL **Location:**

lib/context/global/user.context.tsx:54,

lib/ui/screen-components/wallet/view/main/index.tsx:71, 90, 102,

lib/hooks/useNotification.ts:13, lib/hooks/useChat.tsx:31 **Description:**

Most queries force network-only, and several wallet refetch queries duplicate the same user-id-keyed requests. There is no normalized cache reuse across screens. **Impact:** Redundant network requests and slower screen transitions; the wallet screen fires 4 queries on mount plus 4 refetch queries after a withdrawal. **Reproduction / How to verify:** Open Wallet → 4 simultaneous network-only requests; submit withdrawal → 4 more. **Recommended fix:** Use cache-and-network where freshness matters and default cache reads elsewhere; consolidate the store-profile/wallet queries.

[MEDIUM] usePrintOrder does a linear scan over all orders to find one

Category: Performance / GraphQL **Location:** lib/hooks/usePrintOrder.ts:102-

108 **Description:** To print, the code finds the order by id via

data.restaurantOrders.find(...) over the full in-memory list. The code itself flags this as unoptimized. **Evidence:**

```
/** Current approach is very unoptimized and will slow down drastically if store order exceeds certain number. */  
const order = data.restaurantOrders.find((order: IOrder) => order._id === id);
```

Impact: O(n) per print and reliance on the full list being loaded; degrades as order volume grows. **Reproduction / How to verify:** Print with a large order list; note it depends on the entire restaurantOrders array being present. **Recommended fix:** Add a get-order-by-id query or index orders by id in a map.

[MEDIUM] expo-av used for audio is deprecated in SDK 54

Category: Performance / Native / Dependency **Location:**

lib/context/global/sound.context.tsx:2; also

lib/ui/splash-screen/SplashVideo.tsx,

lib/ui/useable-components/splash/SplashVideo.tsx **Description:** expo-av is deprecated as of SDK 54 (superseded by expo-audio / expo-video). It also re-creates the sound object on each new-order trigger. **Evidence:** import { Audio } from "expo-av"; and Audio.Sound.createAsync(require("@/lib/assets/sound/beep3.mp3")) on every new-order play. **Impact:** Future SDK upgrades will break; repeated load/unload of a 470 KB mp3 adds overhead. **Recommended fix:** Migrate to expo-audio; preload the sound once and replay.

[LOW] Large unoptimized app icon asset

Category: Performance / Assets **Location:** lib/assets/images/icon.png (831 KB); lib/assets/sound/beep3.mp3 (470 KB); lib/assets total 4.4 MB **Description:** The app icon PNG is 831 KB — large for an icon (typically <100 KB). **assetBundlePatterns:** ["**/*"] bundles everything. **Impact:** Slightly larger app download/bundle. **Recommended fix:** Compress the icon (pngquant/zopfli) and scope assetBundlePatterns.

[LOW] moment.js (heavy, maintenance-mode) used for date formatting

Category: Performance / Dependency **Location:** lib/ui/useable-components/order/index.tsx:7 and 2 other files **Description:** moment is in maintenance mode and large; used in a few places for formatting. **Impact:** Extra bundle weight vs. lightweight alternatives. **Recommended fix:** Replace with date-fns/dayjs or native Intl.

[LOW] Inline render functions / no list-item memoization

Category: Performance / React Native Render **Location:** lib/ui/screen-components/home/orders/main/new-orders.tsx:152-188 **Description:** renderItem, ListEmptyComponent, and per-item handlers are defined inline, and Order is not memoized; the dynamic NativeWind classes h- [\${height}px] mb- [\${marginBottom}px] (line 142) are also not statically extractable and won't apply. **Impact:** Avoidable re-renders of list rows during polling/refresh; ineffective dynamic classNames. **Recommended fix:** React.memo(Order), hoist renderItem/keyExtractor, and use style={{ height, marginBottom }} instead of template-literal classes.

[INFO] keep-awake forces the screen on indefinitely

Category: Performance / Native **Location:** app/_layout.tsx:55 (useKeepAwake()) **Description:** The whole app keeps the device awake. This is plausibly intentional (a counter at the till needs the screen on for new-order alerts) but is a battery consideration. **Recommended fix:** Confirm it is required; scope to the active-orders screen if possible.

PART 4 — Irrelevant / Dead Code & Cleanup

[HIGH] Entirely unused GraphQL mutation files (customer/rider leftovers)

Category: Cleanup / Dead Code **Location:**

- `lib/apollo/mutations/authentication.mutation.ts` (login, resetPassword, changePassword, forgotPassword, sendOtpToEmail/Phone, emailExist, phoneExist, Deactivate)
- `lib/apollo/mutations/user.mutation.ts` (createUser, updateUser)
- `lib/apollo/mutations/address.mutation.ts` (deleteAddress, ...)
- `lib/apollo/mutations/activity.mutation.ts` (createActivity)
- `lib/apollo/mutations/push-token.mutation.ts` (pushToken)
- `lib/apollo/mutations/notification.mutation.ts` (updateNotificationStatus) **Description:** None of these files is imported anywhere in `lib/app/` (verified: `grep -rl "mutations/<file>"` returns no importer). They are copy-paste leftovers from the customer/rider apps. **How to verify safely:** `grep -rn "authentication.mutation\|user.mutation\|address.mutation\|activity.mutation\|push-token.mutation\|notification.mutation" lib app` → only the files themselves. **Removal safety:** Safe.

[MEDIUM] Unused individual operations inside live files

Category: Cleanup / Dead Code **Location:**

- `lib/apollo/mutations/order.mutation.ts` — `ASSIGN_ORDER`, `UPDATE_ORDER_STATUS_RIDER`, `reviewOrder`, `placeOrder` (0 importers)
- `lib/apollo/mutations/rider.mutation.ts` — `UPDATE_LOCATION`, `UPDATE_LICENSE`, `UPDATE_VEHICLE` (0 importers; only `UPDATE_AVAILABILITY` and `UPDATE_BUSINESS_DETAILS` are used)
- `lib/api/graphql/mutation/restaurant/index.ts` — `TOGGLE_AVAILABILITY` (0 importers; availability uses `UPDATE_AVAILABILITY/toggleStoreAvailability` instead)
- `lib/apollo/queries/version.query.ts` — `versions`, `getVersions` (only re-exported in the barrel; no force-update logic) **How to verify safely:** Per-symbol `grep` across `lib/app` excluding the definition file → zero hits. **Removal safety:** Safe.

[MEDIUM] Duplicate orders query `STORE_ORDERS`

Category: Cleanup / Duplicate Code **Location:** `lib/apollo/queries/store.query.ts`

(STORE_ORDERS) vs `lib/apollo/queries/orders.ts` (GET_ORDERS) **Description:** STORE_ORDERS is a near-identical copy of GET_ORDERS; it is referenced **only in a commented-out block** in `lib/hooks/useNotification.ts:87`. GET_ORDERS is the live query. **How to verify safely:** `grep -rn "STORE_ORDERS" lib app` → only its definition and a comment. **Removal safety: Safe.**

[MEDIUM] Storage-key inconsistency for language (`lang` vs `enatega-language`)

Category: Cleanup / Correctness **Location:** `i18next.ts:78` reads `enatega-language`; `lib/ui/screen-components/home/language/view/main/index.tsx:36,45`, `lib/context/global/auth.context.tsx:43`, `lib/ui/layouts/root-user/index.tsx:22` read/write `lang` **Description:** The language selector persists under `lang`, but `i18next`'s initial load reads `enatega-language` (never written). The saved language is not reliably restored on cold start. **How to verify safely:** Change language, kill the app, relaunch → initial language may not match the saved one. **Removal safety: Medium risk** (touches `i18n` init — standardize on one key and test all locales).

[LOW] Backup, empty, and stale files

Category: Cleanup / Dead Code **Location:**

- `lib/ui/screen-components/home/drawer/drawer-header/index.tsx.backup` — committed editor backup
- `lib/utils/dummy/index.ts` — 0 bytes / empty
- `tsconfig.json`: `include` lists `lib/hooks/useLogin.js` which does not exist (the file is `useLogin.ts`)
- `google-service-account-copy.json` — duplicate of the secret file (delete after rotation) **Removal safety: Safe** (except the secret file — rotate first, then purge from history).

[LOW] Commented-out dead code blocks

Category: Cleanup / Dead Code **Location:**

- `lib/apollo/index.ts:84-92` — old `terminatingLink` block
- `lib/utils/constants/languages.ts:1-46` — old `LANGUAGES` array commented out
- `lib/hooks/useNotification.ts:86-97` — commented STORE_ORDERS query + navigation
- `app/(protected)/_layout.tsx:38-41` — commented bank-management screen
- `lib/ui/screens/login/index.tsx:181-196` — commented old login button **Removal safety: Safe.**

[HIGH] Installed-but-unused dependencies

Category: Cleanup / Dependency **Location:** `package.json` **Description:** The following packages have **zero import sites** in `lib//app/` (verified by `glob-safe grep` for `'pkg' / "pkg/"`): | Package | Notes | |---|---| | `react-native-webview` | No `WebView` usage anywhere | | `react-native-maps` | No `map` usage in store app | | `react-native-maps-directions` | No usage | | `react-native-ping` | No usage | | `react-native-countdown-component` (+ `@types/react-native-countdown-component`) | Custom `custom-timer` used instead | | `lottie-react-native` | No `Lottie` usage | | `react-native-thermal-receipt-printer-image-qr` | Only `react-native-thermal-printer` is imported | | `graphql-ws` | Installed but `WS` link uses deprecated | | `subscriptions-transport-ws` | | `expo-linear-gradient` | No `LinearGradient` usage | | `react-native-get-random-values` | Not imported as a polyfill anywhere (verify before removal) | | `ajv` | No direct import (verify it isn't required by a build step) | **How to verify safely:** For each, `grep -rn "<pkg>" lib app` → no source import. Remove, reinstall, run a full build + smoke test (printing, subscriptions, splash). **Removal safety: Medium risk** — confirm none are required transitively at build time (`react-native-get-random-values`, `ajv`) before deleting; the clearly-unused UI/native ones (`react-native-maps*`, `react-native-webview`, `lottie-react-native`, `react-native-ping`, `react-native-countdown-component`, `expo-linear-gradient`, `react-native-maps-directions`, the duplicate `thermal printer`) are **Safe**.

[INFO] react-native-worklets / react-native-reanimated

Category: Cleanup / Dependency (info) **Description:** `react-native-worklets` shows 0 direct imports but is a required runtime for `Reanimated v4` (driven by the Babel preset). **Do not remove.** Listed here to prevent an incorrect cleanup.

Prioritized Remediation Roadmap

Immediate (do today — secrets & credential exposure)

1. **Rotate** the Firebase Admin SDK key, the Google Play publishing key, the Sentry auth token, and (if API restrictions aren't set) the Firebase API key.
2. **Delete** `google-service-account.json`, `google-service-account-copy.json` from the tree and **purge from git history**; move to EAS secrets.
3. **Fix .gitignore** (`google-servicees.json` → `google-services.json`; add `google-service-account*.json`).
4. **Remove password** from `STORE_PROFILE` and the profile UI; stop returning passwords from the backend.
5. **Disable demo credential auto-fill** (`lastOrderCreds`) in production; have the backend

stop returning a real password.

6. **Strip sensitive console.*** (login/token logs) and add a release log-stripper.

Short-term (this sprint)

1. Fix the subscription key mismatch (`restaurantId` → `store-id/userId`) and then **reduce polling** from 5s and drop `network-only`.
2. Add `[orderId]` deps to the chat subscription effect; remove the `orders.length===0` refetch effect.
3. Add an auth guard to `(protected)/_layout.tsx` and auto-logout on `UNAUTHENTICATED`.
4. Add `disabled={isLoading}` to `CustomContinueButton`; guard accept/withdraw handlers; confirm backend idempotency.
5. Standardize the language storage key; initialize `i18next` once; ship only supported locales.
6. Register `eslint-plugin-react-hooks` (rules-of-hooks: error) and add `no-console`.

Larger refactors

1. Migrate WebSocket transport to `graphql-ws` + `GraphQLWSLink`; remove `subscriptions-transport-ws`.
2. Re-architect store identity to come from the JWT (not `AsyncStorage`) for all owner-scoped operations; coordinate BOLA hardening with the backend.
3. Migrate `expo-av` → `expo-audio`; consolidate query/cache strategy; memoize list items.
4. Remove dead GraphQL files/operations and unused dependencies; add a CI secret scanner (`gitLeaks`) and `depcheck`.

Safe Cleanup Checklist

- ☐ Remove unused file:
`lib/apollo/mutations/authentication.mutation.ts`
- ☐ Remove unused file: `lib/apollo/mutations/user.mutation.ts`
- ☐ Remove unused file: `lib/apollo/mutations/address.mutation.ts`
- ☐ Remove unused file: `lib/apollo/mutations/activity.mutation.ts`
- ☐ Remove unused file: `lib/apollo/mutations/push-token.mutation.ts`
- ☐ Remove unused file: `lib/apollo/mutations/notification.mutation.ts`
- ☐ Remove unused query file: `lib/apollo/queries/version.query.ts`

- ☐ Remove unused mutations: `ASSIGN_ORDER`, `UPDATE_ORDER_STATUS_RIDER`, `reviewOrder`, `placeOrder` (`order.mutation.ts`)
 - ☐ Remove unused mutations: `UPDATE_LOCATION`, `UPDATE_LICENSE`, `UPDATE_VEHICLE` (`rider.mutation.ts`)
 - ☐ Remove unused mutation: `TOGGLE_AVAILIBILITY` (`api/graphql/mutation/restaurant/index.ts`)
 - ☐ Remove duplicate query: `STORE_ORDERS` (`store.query.ts`)
 - ☐ Remove backup file: `lib/ui/screen-components/home/drawer/drawer-header/index.tsx.backup`
 - ☐ Remove empty file: `lib/utils/dummy/index.ts`
 - ☐ Fix `tsconfig.json` stale include: `lib/hooks/useLogin.js`
 - ☐ Remove commented code in: `lib/apollo/index.ts`, `lib/utils/constants/languages.ts`, `lib/hooks/useNotification.ts`, `app/(protected)/_layout.tsx`, `lib/ui/screens/login/index.tsx`
 - ☐ Remove unused dependency: `react-native-webview`
 - ☐ Remove unused dependency: `react-native-maps`
 - ☐ Remove unused dependency: `react-native-maps-directions`
 - ☐ Remove unused dependency: `react-native-ping`
 - ☐ Remove unused dependency: `react-native-countdown-component` (+ `@types/...`)
 - ☐ Remove unused dependency: `lottie-react-native`
 - ☐ Remove unused dependency: `react-native-thermal-receipt-printer-image-qr`
 - ☐ Remove unused dependency: `expo-linear-gradient`
 - ☐ Replace then remove deprecated transport dependency: `subscriptions-transport-ws` (use `graphql-ws`)
 - ☐ Confirm before deleting (build-time/transitive use): `react-native-get-random-values`, `ajv`
 - ☐ Confirm before deleting (DO NOT remove): `react-native-worklets` (required by Reanimated v4)
 - ☐ Confirm before deleting (secret — rotate first, then purge): `google-service-account copy.json`, `google-service-account.json`
-

Developer Implementation Checklist

Security

- ☐ Rotate all four leaked secrets (Firebase Admin, Play publisher, Sentry, Firebase API key).
- ☐ Purge secret files from git history; move to EAS secrets; fix `.gitignore`.
- ☐ Remove `password` from `STORE_PROFILE` + profile UI.
- ☐ Gate/remove `lastOrderCreds` demo auto-fill in production.
- ☐ Remove sensitive `console.*`; add `no-console` + `babel-plugin-transform-remove-console`.
- ☐ Add (protected) auth guard + auto-logout on auth errors.
- ☐ Coordinate BOLA fix: server derives store identity from JWT.

Performance

- ☐ Fix `restaurantId` → `store-id` subscription key; cut polling interval; drop `network-only`.
- ☐ Add `deps` array to chat subscription effect; delete `refetch-on-empty` effect.
- ☐ Memoize `Order`; hoist `renderItem/keyExtractor`; fix `dynamic className` → `style`.
- ☐ Lazy-load locales / ship only supported set; single `i18next` init.
- ☐ Migrate `expo-av` → `expo-audio`; preload sound.

Cleanup / Dependencies

- ☐ Execute the Safe Cleanup Checklist above; run `depcheck/build` after removal.
- ☐ Add `eslint-plugin-react-hooks`; re-enable `exhaustive-deps`.
- ☐ Add `git leaks` pre-commit + CI.

Testing

- ☐ Full regression on login/logout, order accept/print, withdraw, language switch after cleanup.

QA Test Plan Checklist

- ☐

Login (valid)

- Screen/Component: `lib/ui/screens/login`, `useLogin`
- Steps: Enter valid store creds → Login.

- Expected: Token stored in SecureStore (store-token), store-id in AsyncStorage, redirect to home.
- Evidence: SecureStore/AsyncStorage dump; network RestaurantLogin 200.

• ☐

Login (invalid) — user enumeration

- Steps: Wrong username vs wrong password.
- Expected: Identical generic error; no "user not found" vs "wrong password" distinction.
- Evidence: FlashMessage text for both cases.

• ☐

Demo credential auto-fill

- Steps: Fresh install → open Login.
- Expected (after fix): fields empty in production.
- Evidence: screenshot; LastOrderCreds response.

• ☐

Logout

- Steps: Profile → Logout.
- Expected: store-token deleted, store-id removed, Apollo store cleared, redirect to login.
- Evidence: storage dump shows keys gone.

• ☐

Expired/invalid token

- Steps: Replace store-token with garbage; relaunch / make a request.
- Expected: Redirect to login (after guard fix); no protected data rendered.
- Evidence: network 401/UNAUTHENTICATED; navigation state.

• ☐

Protected screen access without auth

- Steps: Clear token while on a protected tab.
- Expected: Guard redirects to login.
- Evidence: screen recording.

• ☐

Token storage inspection

- Steps: Inspect SecureStore + AsyncStorage + Apollo cache.
- Expected: No plaintext password anywhere; tokens only in SecureStore.
- Evidence: storage/cache dump (password must be absent after fix).
- ☐

Vendor accessing another vendor's store/earnings (BOLA)

- Steps: Swap `store-id` to another store's id (rooted/instrumented).
- Expected: Backend rejects (data scoped to JWT).
- Evidence: Restaurant/StoreEarnings responses for the swapped id.
- ☐

Vendor creating withdrawal for another store

- Steps: Swap `store-id`; submit withdrawal.
- Expected: Rejected by backend.
- Evidence: `createWithdrawRequest` response.
- ☐

Availability toggle for another store

- Steps: Swap `store-id`; toggle availability.
- Expected: Rejected by backend.
- Evidence: `toggleStoreAvailability` response.
- ☐

Order accept/reject manipulation

- Steps: Call accept/cancel with an `_id` from a different store.
- Expected: Backend rejects (not your order).
- Evidence: `acceptOrder/cancelOrder` response.
- ☐

Skipping order workflow steps

- Steps: Attempt PICKED before ACCEPTED, etc.
- Expected: Backend enforces valid transitions.
- Evidence: mutation responses.
- ☐

Duplicate order status update (double-tap)

- Screen/Component: `set-order-accept-time`, `CustomContinueButton`

- Steps: Double-tap "Done"/"Accept and Print".
- Expected (after fix): button disabled while in-flight; single `acceptOrder` call.
- Evidence: network shows one request.

• ☐

Image upload file-type/size validation

- Expected: N/A — no upload feature exists (document as not applicable).

• ☐

Deep link abuse

- Steps: Trigger `myapp://` deep links.
- Expected: No unauthenticated access to protected routes.
- Evidence: navigation behavior.

• ☐

Unsafe WebView/HTML rendering

- Expected: N/A — no WebView/HTML rendering (document as not applicable).

• ☐

Large order list performance

- Steps: Load 100+ orders.
- Expected: Smooth scroll; no per-5s full-list jank after polling fix.
- Evidence: FPS/perf monitor; network cadence.

• ☐

Large menu/product list performance

- Expected: N/A — store app has no product-management list (orders only); document.

• ☐

Image upload performance

- Expected: N/A — no upload.

• ☐

Duplicate API call check

- Steps: Idle on New Orders with no orders.
- Expected (after fix): only the poll/subscription, no extra refetch loop.
- Evidence: network log.

• ☐

Removed-code regression

- Steps: After cleanup, full smoke test (login, accept, print, withdraw, language, chat).
 - Expected: No broken imports; app builds and runs.
 - Evidence: build log + manual pass.
-

Dependency Upgrade Plan

Live `npm audit` could not be run during this audit (no network). CVE confirmation should be done with `npm audit / osv-scanner` against `package-lock.json` in CI. The items below are based on declared versions and maintenance status verified from `package.json`.

Package	Current	Concern	Recommended action	Used in code?
subscription-s-transport-ws	^0.11.0	Deprecated/unmaintained (since ~2021)	Replace with <code>graphql-ws</code> + <code>GraphQLWSLink</code>	Yes (via WS link)
graphql-ws	^6.0.6	Installed but unused	Adopt (replaces above)	No
expo-av	~16.0.7	Deprecated in SDK 54	Migrate to <code>expo-audio/expo-video</code>	Yes (3 files)
moment	^2.30.1	Maintenance mode, heavy	Migrate to <code>date-fns/dayjs</code>	Yes (3 files)
react-native-webview	13.15.0	Unused	Remove	No
react-native-maps (+ -directions)	1.20.1 / ^1.9.0	Unused	Remove	No
lottie-react-native	~7.3.1	Unused	Remove	No
react-native-ping	^1.2.8	Unused	Remove	No
react-native-countdown-component	^2.7.1	Unused (custom timer used)	Remove (+ types)	No
react-native-thermal-	^0.1.12	Unused duplicate	Remove	No

Package	Current	Concern	Recommended action	Used in code?
receipt-printer-image-qr				
expo-linear-gradient	~15.0.7	Unused	Remove	No
react-native-get-random-values	~1.11.0	No import found	Verify, then remove	No (verify)
ajv	^8.17.1	No direct import	Verify build need, then remove	No (verify)
All expo- [*] / RN core	SDK 54 / RN 0.81.5	Current	Keep; run expo-doctor after cleanup	Yes

Final Risk Rating

CRITICAL

The rating is driven by **directly verifiable, high-impact secret exposure and credential handling**, not speculation:

1. A **Firebase Admin SDK private key** and a **Google Play publishing private key** are committed and tracked in git (the latter not even covered by `.gitignore`), and both are present in git history. These enable full backend compromise and malicious app distribution respectively, and require key rotation regardless of remediation speed.
2. The **store's plaintext password is fetched from the API and rendered in the profile UI**, implying recoverable server-side password storage and enabling trivial credential capture.
3. A **Sentry auth token** is hardcoded in `eas.json`, and the **Firebase API key / google-services.json** is committed with a typo'd ignore rule.
4. Supporting weaknesses — `store-id`-based authorization (BOLA surface, backend-dependent), sensitive console logging, and no protected-route guard — compound the exposure.

The performance and cleanup issues (broken realtime subscription forcing 5s `network-only` polling, re-subscribing chat effect, 31 eager locale bundles, ~11 unused dependencies, six dead GraphQL files) are meaningful for cost, battery, and maintainability but are secondary to the secret/credential findings. **Once all four committed secrets are rotated and purged and the password field is removed, the residual rating would drop to High** (pending backend confirmation of JWT-based authorization), and to Medium after the BOLA and logging items are

closed.